醫工三 B12508045 盧雅筠

Theoretical questions

1. A common measure of transmission for digital data is the baud rate, defined as the number of bits transmitted per second. Generally, transmission is accomplished in packets consisting of a start bit, a byte (8 bits) of information, and a stop bit. Using these facts, answer the following:
(a) How many minutes would it take to transmit a 1024 X 1024 image with 256 intensity levels using a 56K baud modem? (10%)
(b) What would the time be at 3000K baud, a representative medium speed of a phone DSL (Digital Subscriber Line) connection? (5%)
數位用戶線路

1. (a) 256 intensity level = 2^8 ⇒ 8 bits = 1 byte / pixel

1024 x 1024 (pixel) x 1(byte/pixel) = 1048576 bytes

a packet = 1 bit for a start bit + 8 bits for information + 1 bit for a stop bit = 10 bits

1048576 x 10 = 10485760 bits , 56k baud moderm = 56k bits / second

$\frac{10485760}{56000} \div 60 \fallingdotseq 3.12 \text{min}$ #

(b) same as (a) to transmit 1024x1024 image need 10485760 bits , 3000k baud = 3000k bit / second

$\frac{10485760}{3000000} \fallingdotseq 3.50\text{s}$ #

2. Consider the image segment shown in below figure. Compute length of the shortest-4, shortest-8 & shortest-m paths between pixels p & q where, V = {1,2} and explain why? (10%)

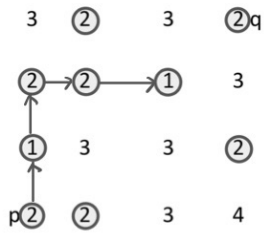
3	②	3	②q
②	②	①	3
①	3	3	②
p②	②	3	4

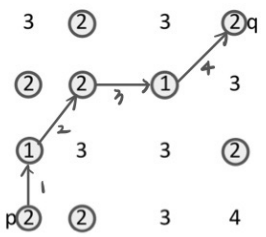
2. pixel value ∈ {1,2,3,4}, V = {1,2} ⇒ 可走的pixel, {3,4}為障礙

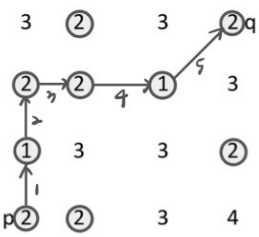
(a) shortest-4 : 只能走上下左右 ↑↓←→ (b) shortest-8 : 能走上下左右,斜 ↖↑↗←→↙↓↘ (c) shortest-m :

{ - 般走 ↑↓←→ 4-鄰接

對兩兩點的共同4-鄰居中沒有在V, 才能斜走 ↗↘







無法從p走到q ⇒ length is ∞ #

斜走可縮短length ⇒ lenght=4 #

length= 5 #

Programming exercises (Send the source code)

1. 程式碼為 Figure 1，輸出的結果為 Figure 2。

```python
import cv2
import numpy as np
from math import sqrt

input_path  = "./Fig1.bmp"
output_path = "./Fig1_triangle.bmp"
top = (150, 150)
side = 200
gray = 255
thickness = 2

img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)

x0, y0 = top
h = (sqrt(3)/2.0) * side
T = (int(round(x0)), int(round(y0)))
L = (int(round(x0 - side/2.0)), int(round(y0 + h)))
R = (int(round(x0 + side/2.0)), int(round(y0 + h)))
pts = np.array([T, L, R], dtype=np.int32)

cv2.polylines(img, [pts], isClosed=True, color=gray, thickness=thickness)
cv2.imwrite(output_path, img)
```

Figure 1

Figure 2

2. 程式碼為 Figure 3，輸出的結果為 Figure 4。

```python
import cv2, numpy as np

input_path  = "./Fig1.bmp"
img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) if len(img.shape)==3 else img

N = int(input("Enter gray levels (power of 2, e.g. 2/4/8/.../256): ").strip())
if not (2 <= N <= 256 and (N & (N-1)) == 0):
    raise ValueError("N must be the power of 2 and in range [2, 256]")

# Quantization
step = 256 // N
q = (gray // step) * step
q = q.astype(np.uint8)

output_path = f"Fig1_quantized_L{N}.bmp"
cv2.imwrite(output_path, q)
```

Figure 3

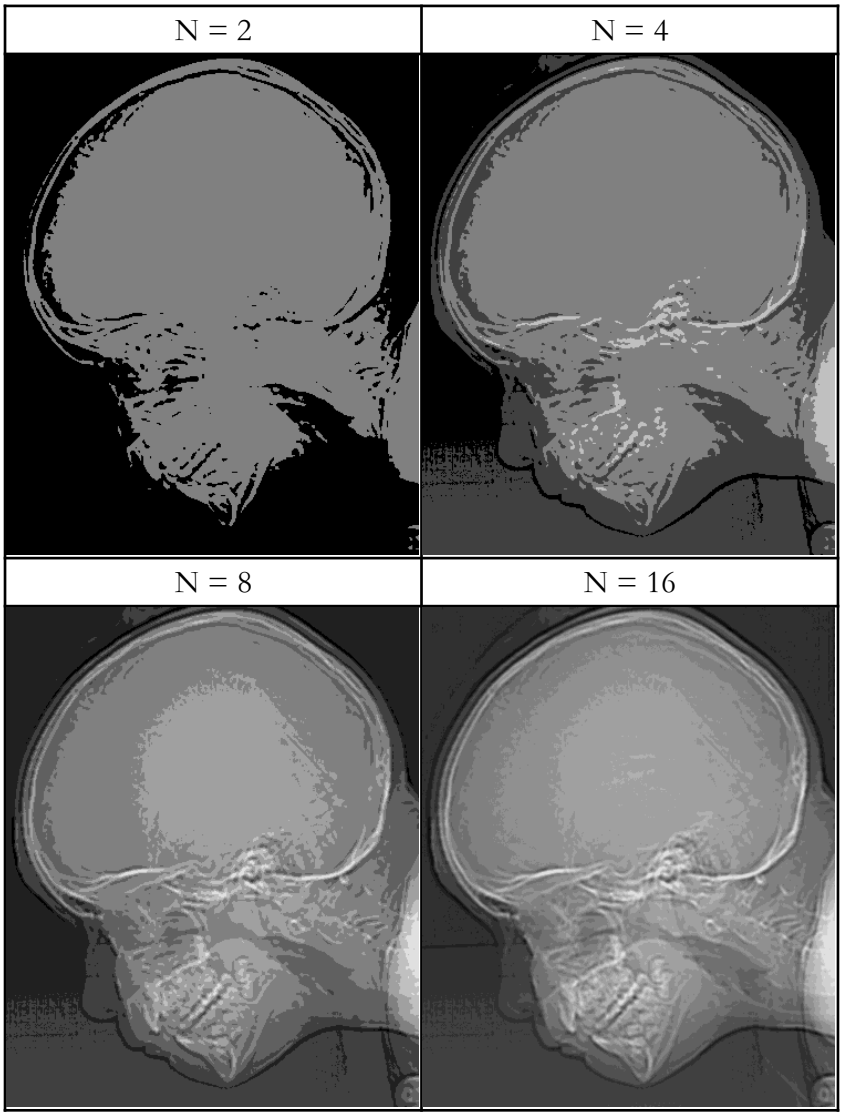


Figure 4

3. 程式碼為 Figure 5，輸出的結果為 Figure 6。

```python
import cv2
import numpy as np

input_path  = "./Fig2.bmp"
img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)

h, w = img.shape[:2]
cx, cy = (w - 1) / 2.0, (h - 1) / 2.0   # center

angles = [15, 30, 45, 60, 90]
for ang in angles:
    M = cv2.getRotationMatrix2D((cx, cy), -ang, 1.0) # clockwise
    out = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_NEAREST, borderValue=0)
    cv2.imwrite(f"Fig2_rot_{ang}deg.bmp", out)
```

Figure 5

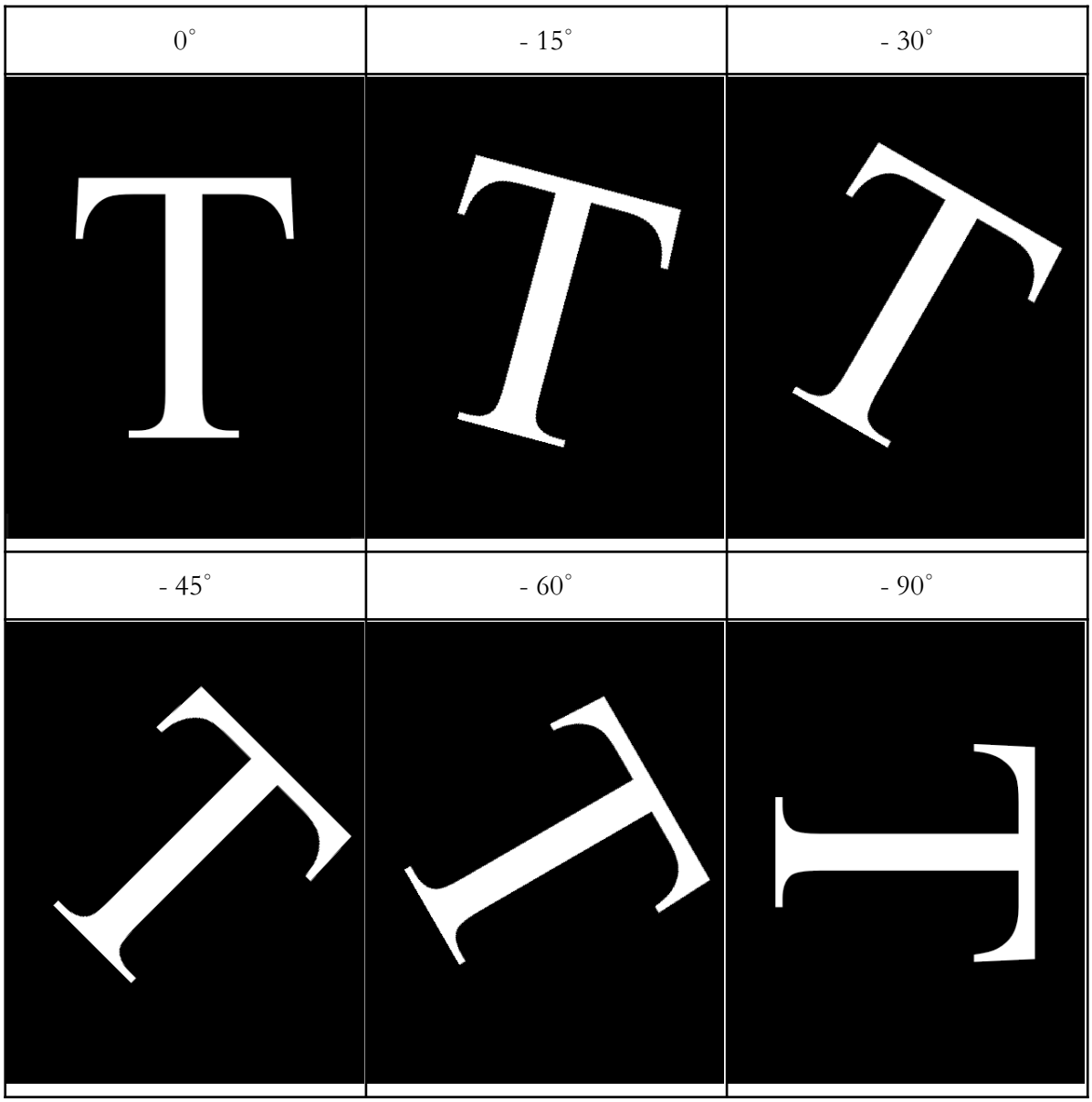


Figure 6

4. 程式碼為 Figure 7，輸出的結果為 Figure 8。

```python
import cv2
import numpy as np
import os

input_path  = "./Fig3.GIF"
output_fold = "results_hw1_4"
os.makedirs(output_fold, exist_ok=True)

img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) if len(img.shape) == 3 else img

# CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(12,12))
enh   = clahe.apply(gray)
cv2.imwrite(f"{output_fold}/clahe.png", enh)

# Gaussian adaptive thresholding
block = 41 # must be odd
C     = 10

bin_gauss = cv2.adaptiveThreshold(enh, 255,
                                  cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                  cv2.THRESH_BINARY,
                                  block, C)
cv2.imwrite(f"{output_fold}/bin_gauss.png", bin_gauss)
```

Figure 7

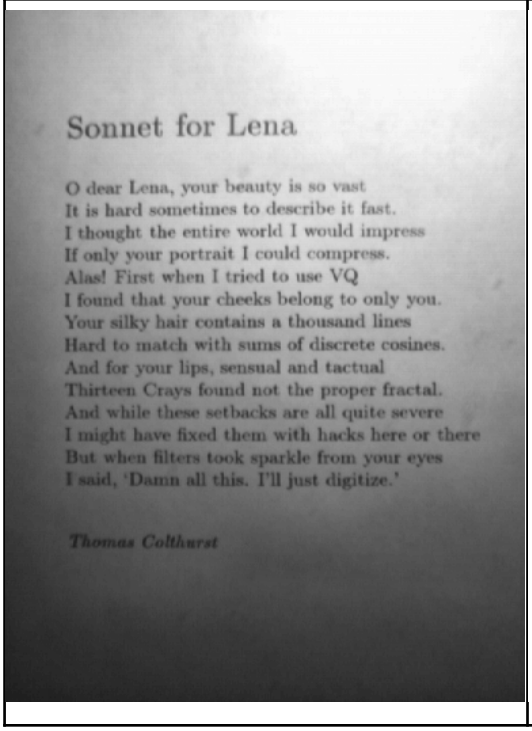
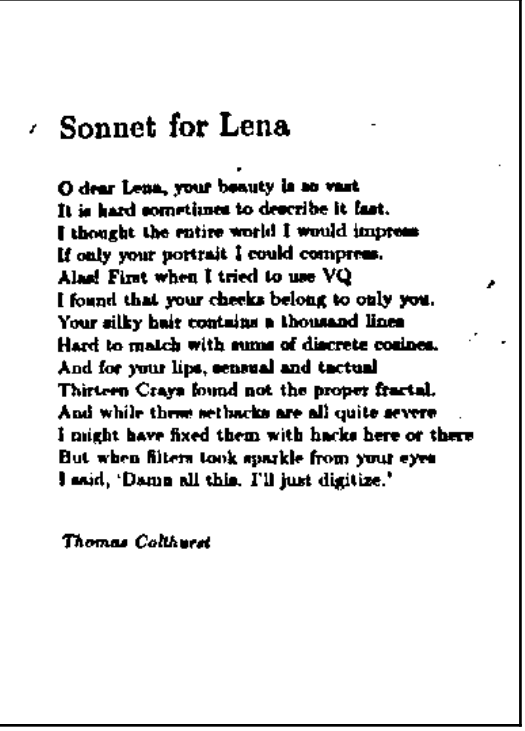
CLAHE	Gaussian adaptive thresholding
	

Figure 8